```
#!/usr/bin/env coffee

coffee = require 'coffee-script'
fs     = require 'fs'

SPEC_PATH = './spec/'
SRC_PATH = './src/'

test = (file) ->
  fs.readFile SPEC_PATH + file, 'utf-8', (err, data) ->
    it = """
      {fact} = require '../fact'
      assert = require 'assert'
      #{data}
    """
    coffee.run it, filename: SPEC_PATH + file

spec = (file) ->
  if /#/.test file then false
  else /\.spec\.coffee$/.test file


tests = ->
  fs.readdir SPEC_PATH, (err, files) ->
    for file in files
      test file if spec file

fs.watch SPEC_PATH, (event, filename) ->
  tests()

fs.watch SRC_PATH, (event, filename) ->
  tests()
```

Specify that this shell script is CoffeeScript.

Define some paths to use everywhere in the program. These are uppercase by convention only; there are no real constants in CoffeeScript.

Add the file path back to coffee.run so it knows where to look for requires inside the file.

Watch the specifications and the program source. Rerun all the tests if any files change in either of them.

This `autotest` runs as a script in a *nix environment (such as your MacBook or your Linux or BSD server). Consult a local expert for how to create a version for your operating system.

The test script and watch script provided here read only a single directory. In practice you might need to monitor all of the subdirectories as well. But it's just as likely that you'll be using an existing testing framework. Having seen how the testing stack goes together, though, you're in a better position to work with an existing testing framework. You know how to use a test framework because you've written one. That's right: in this chapter you've written your own test framework. It wasn't that difficult!

By writing (or using) a test framework, you've made it easier to write tests. If it's easier to write tests, it's more likely that you actually *will* write tests and, as a result, actually have evidence that your program behaves the way